

Bayes: la transición entre discriminativo y generativo

MMIA 6013 — IA Generativa y Agentes · Sesión 01 · 14 de septiembre de 2026

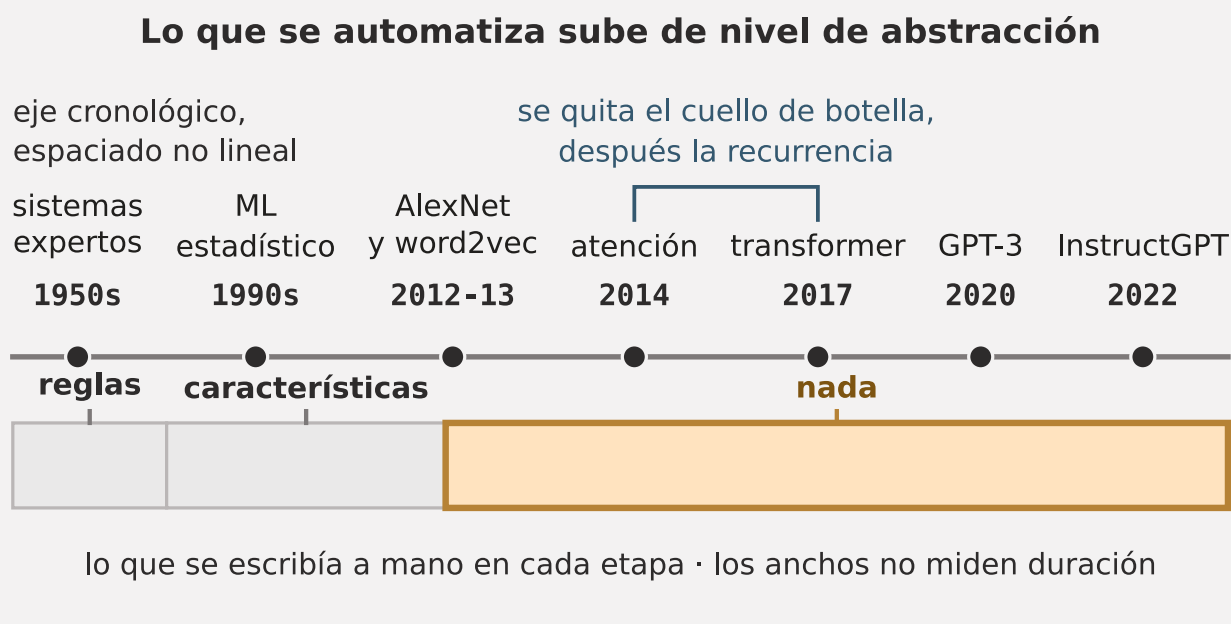
Daniel Andrés Riofrío Almeida

La IA antes de los LLMs

01

Lo que se escribía a mano fue subiendo de nivel

- Simbólico (1950s–80s): MYCIN, unas 600 reglas if-then escritas por expertos. Lección: mantener reglas a mano no escala
- ML estadístico (1990s–2000s): modelos ocultos de Markov (HMM), SVM, redes bayesianas. Aprenden de datos, pero con características diseñadas a mano
- Deep learning (2012–2016): AlexNet, RNN/LSTM y word2vec (2013), que aprende qué significa una palabra por las palabras junto a las que aparece. Lección: las representaciones también se aprenden
- En texto el problema no era el cómputo: el codificador recurrente resumía la frase entera en un solo vector y se atascaba con frases largas. La atención (2014) abrió ese cuello
- Y la recurrencia obligaba a procesar palabra por palabra, así que no se podía entrenar en paralelo. El transformer (2017) la quitó; GPT-3 (2020) mostró qué pasa al escalarlo



reglas	MYCIN: unas 600 reglas if-then escritas por expertos.
características	HMM y SVM aprenden de los datos, no la representación.
nada	AlexNet y word2vec aprenden también la representación.

Las técnicas clásicas eran poderosas pero frágiles: escalaron mal porque exigían ingeniería manual.

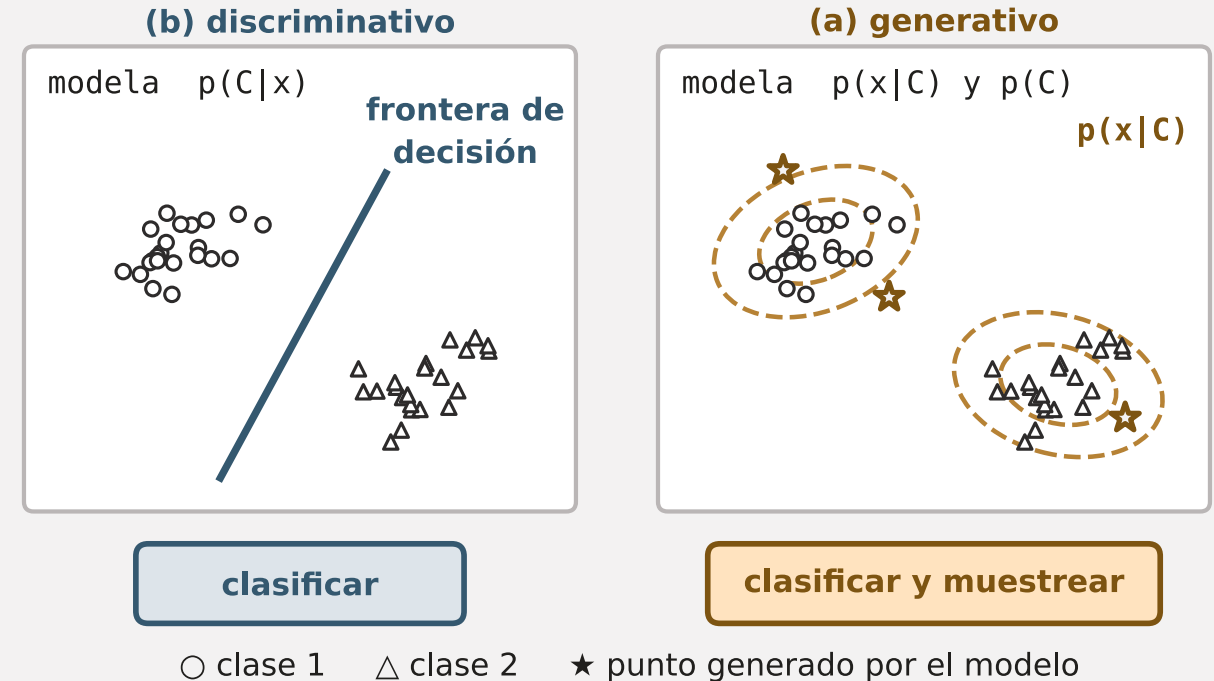
$P(X)$ frente a $P(Y|X)$

01

Bishop parte el problema en tres

- (a) Generativo: modelar $p(x | C_k)$ y $p(C_k)$, y llegar a $p(C_k | x)$ por Bayes; equivale a modelar la conjunta $p(x, C_k)$
- (b) Discriminativo: modelar directamente la posterior $p(C_k | x)$ y después decidir
- (c) Función discriminante: aprender $f(x)$ que va de la entrada a la etiqueta; las probabilidades no juegan ningún papel
- Solo (a) permite muestrear puntos nuevos en el espacio de entrada: eso, y no otra cosa, es ser generativo
- Estrictamente una SVM es (c); en clase agrupamos (b) y (c) bajo ‘discriminativo’, siguiendo a Ng y Jordan
- Generativos: Naive Bayes, mezcla de gaussianas (GMM), HMM, autocodificador variacional (VAE), GPT. Discriminativos: regresión logística, SVM

Se aprenden objetos distintos, no versiones mejor y peor

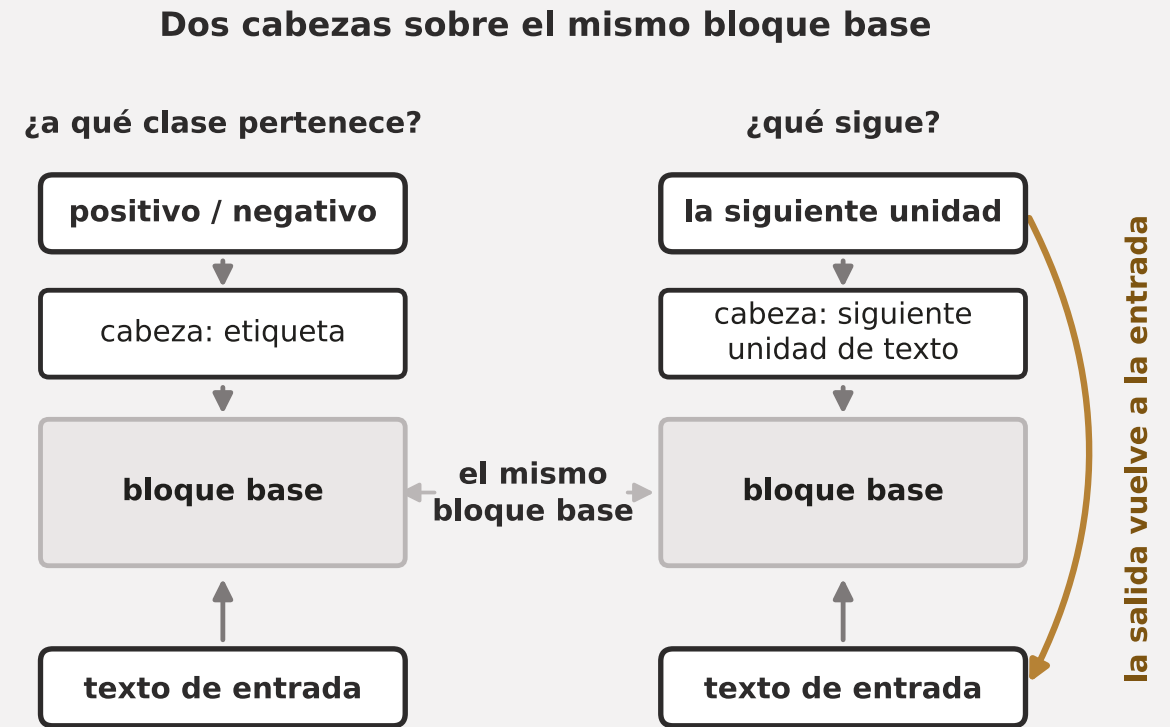


Solo el generativo permite producir un punto nuevo del espacio de entrada.

Un clasificador excelente que no puede producir un dato nuevo no es generativo.

Un BERT afinado para clasificar sentimiento, ¿es generativo o discriminativo?

- A. Generativo: por dentro es la misma familia de arquitectura que GPT
- B. Discriminativo: aprende $P(\text{etiqueta} \mid \text{texto})$ y no puede producir texto nuevo
- C. Depende del tamaño del modelo y de cuántos datos se usaron para afinarlo



Los dos comparten el bloque base; cambian la cabeza y la pregunta que modela cada uno.

El criterio para decidir es la partición de Bishop de la lámina anterior: se lee en voz alta después de recoger las respuestas.

No hay ganador, hay dos regímenes

- La creencia popular, apoyada en Vapnik y compartida por Bishop: si solo quieres decidir la clase, estimar la conjunta es un desperdicio de cómputo y de datos
- Proposición 1: asintóticamente el error del discriminativo es menor o igual que el del generativo. La creencia no es falsa, es incompleta
- A Naive Bayes le bastan $m = \Omega(\log n)$ ejemplos para acercarse a su asíntota; a la regresión logística le hacen falta $m = \Omega(n)$, con n la dimensión de la entrada
- Con pocos datos gana el generativo aunque su asíntota sea peor; con muchos, lo alcanza y lo supera el discriminativo. Las curvas se cruzan
- Lo que $P(X)$ sí compra: muestrear, detectar datos atípicos y entrenar sin etiquetas. Eso es un modelo fundacional: uno preentrenado sobre corpus sin etiquetar, que sirve de base para muchas tareas



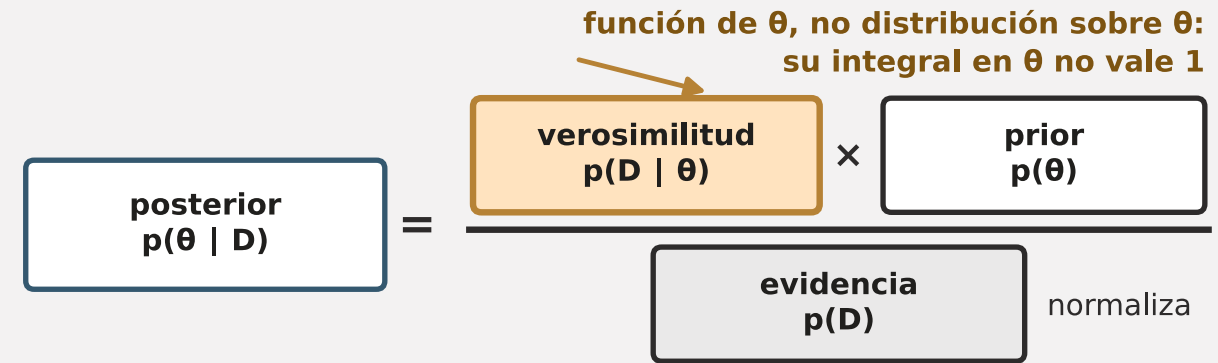
El puente bayesiano

01

posterior $\propto$ verosimilitud $\times$ prior

- $p(\theta \mid D) = p(D \mid \theta) \cdot p(\theta) / p(D)$
- Prior $p(\theta)$: lo que suponemos de los parámetros antes de ver los datos
- Verosimilitud $p(D \mid \theta)$: el conjunto observado, evaluado y leído como función de θ
- Posterior $p(\theta \mid D)$: la incertidumbre sobre θ después de observar D
- Evidencia $p(D) = \int p(D \mid \theta) p(\theta) d\theta$: la constante que normaliza
- Hoy es vocabulario mínimo: lo necesitamos para leer los cuatro hitos del bloque siguiente

Las cuatro piezas no son intercambiables



prior $p(\theta)$	lo que suponemos de los parámetros antes de ver los datos
verosimilitud $p(D \mid \theta)$	el conjunto observado, leído como función de θ
posterior $p(\theta \mid D)$	la incertidumbre sobre θ después de observar D
evidencia $p(D)$	$\int p(D \mid \theta) p(\theta) d\theta$ — la constante que normaliza

Hoy es vocabulario mínimo: lo necesitamos para leer los cuatro hitos del bloque siguiente.

El mismo teorema, dos planos distintos

- Plano de los parámetros: prior $p(\theta)$, verosimilitud $p(D \mid \theta)$, posterior $p(\theta \mid D)$
- Plano de la clase o del latente: prior $p(C_k)$ o π_k , verosimilitud $p(x \mid C_k)$, posterior $p(C_k \mid x)$ o $\gamma(z_k)$
- Regla de higiene: decir siempre de qué es el posterior, si de θ , de la clase o del latente
- Máxima verosimilitud: fijar θ donde $p(D \mid \theta)$ es máxima; su logaritmo negativo es lo que en aprendizaje automático se llama función de error
- Entrenar un modelo sobre un corpus es exactamente eso: θ son los pesos, X es el corpus, y se maximiza $P(X \mid \theta)$. La sesión 03 lo llamará preentrenamiento
- El paso E de esperanza-maximización (EM) es este teorema aplicado al latente: este bloque no es preámbulo del siguiente, es su motor

La ambigüedad está en el lenguaje, no en la matemática

la misma terna de palabras nombra objetos distintos según el plano

	prior	verosimilitud	posterior	dónde aparece hoy
sobre los parámetros	$p(\theta)$	$p(D \mid \theta)$	$p(\theta \mid D)$	entrenamiento sobre el corpus
sobre la clase o el latente	$p(C_k)$ o π_k	$p(x \mid C_k)$	$p(C_k \mid x)$ o $\gamma(z_k)$	Naive Bayes y GMM

misma palabra, dos objetos

Regla de higiene: decir siempre de qué es el posterior.

Máxima verosimilitud: fijar θ donde $p(D \mid \theta)$ es máxima; su logaritmo negativo es lo que en aprendizaje automático se llama función de error.

El paso E de EM es este teorema aplicado al latente: este bloque es el motor del siguiente, no su preámbulo.

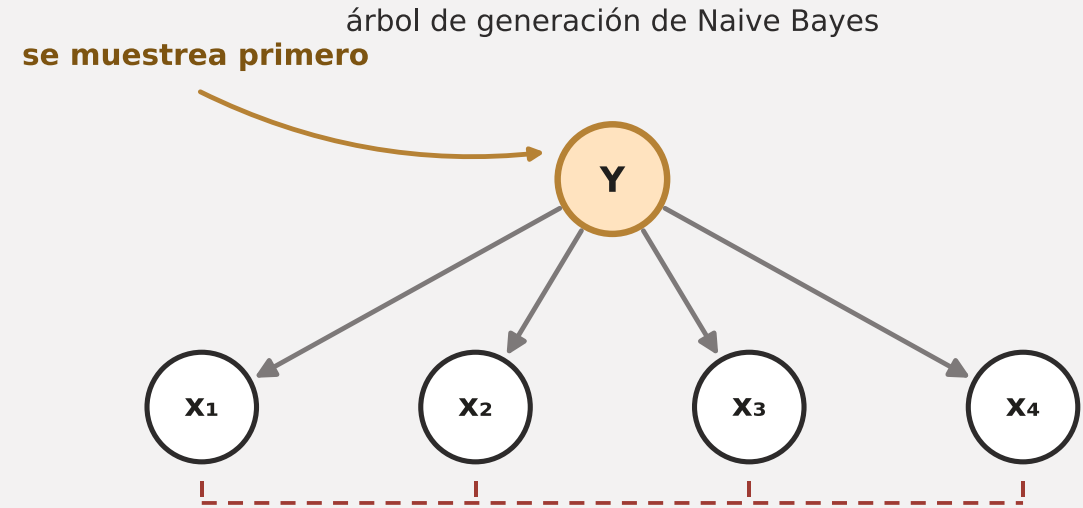
Cuatro factorizaciones de $P(X)$

01

Naive Bayes, el generativo mínimo

- Factoriza: $P(X, Y) = P(Y) \cdot P(x_1 | Y) \cdot \dots \cdot P(x_n | Y)$
- Genera así: primero muestrea la clase Y, luego cada característica de forma independiente dado Y
- Se estima contando: frecuencias con suavizado de Laplace. Literalmente contar
- Filtro de spam: $P(Y=\text{spam})$ es la frecuencia de spam en el corpus; $P(\text{'oferta'}|\text{spam})$, la de esa palabra dentro de los spam
- Su par discriminativo es la regresión logística: misma familia, dos objetivos de ajuste distintos
- Limitación: la independencia condicional entre características casi nunca se cumple en texto

La suposición del modelo es justo lo que no dibuja



aquí faltan las dependencias reales del texto

$$P(X, Y) = P(Y) \cdot P(x_1 | Y) \cdot P(x_2 | Y) \cdot P(x_3 | Y) \cdot P(x_4 | Y)$$

Genera así: primero la clase Y, después cada característica de forma independiente dado Y.

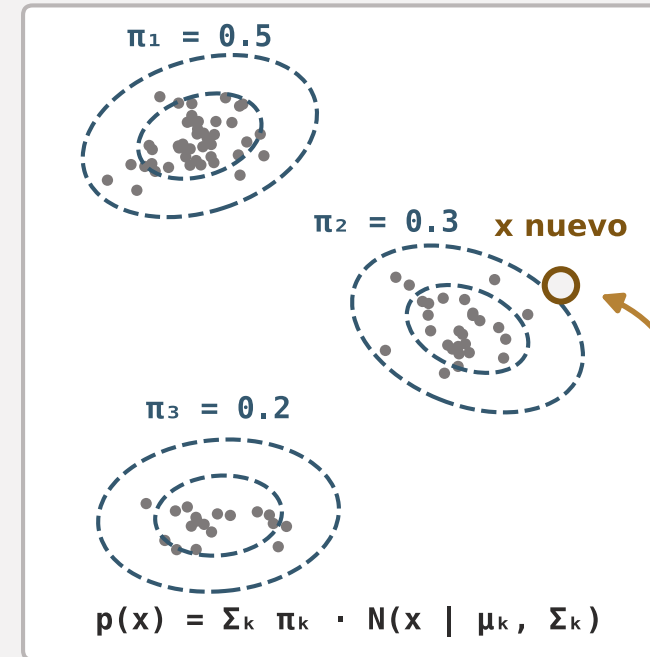
Se estima contando: frecuencias con suavizado de Laplace. Su par discriminativo es la regresión logística: misma familia, dos objetivos de ajuste.

GMM, la variable latente que nunca se observa

- Factoriza: $p(x) = \sum_k \pi_k N(x | \mu_k, \Sigma_k)$, con π_k entre 0 y 1 y $\sum_k \pi_k = 1$
- Con latente explícita: $p(x) = \sum_z p(z)p(x | z)$, donde z es binaria 1-de- K y nunca se observa
- Genera por muestreo ancestral: primero $\hat{z}$ de $p(z)$, después x de $p(x|\hat{z})$. Es lo que hace `gmm.sample()`
- Responsabilidad $\gamma(z_k) = p(z_k=1|x)$: el teorema de Bayes aplicado al latente
- EM alterna: el paso E evalúa las responsabilidades, el paso M reestima μ_k, Σ_k y π_k
- Honestidad: hay máximos locales y singularidades; EM no garantiza el mayor. Y cada punto es independiente: no hay secuencia

Generar es un procedimiento en dos etapas

los puntos grises son los datos observados



muestreo ancestral

paso 1
elegir el componente: $\hat{z} \sim p(z)$.
Aquí sale $k = 2$.

paso 2
generar el punto dentro de ese componente: $x \sim N(\mu_2, \Sigma_2)$.
Sale hueco: no es un dato.

z nunca se observa en los datos: solo se infiere como responsabilidad $\gamma(z_k)$.

$\gamma(z_k) = p(z_k = 1 | x)$ es el teorema de Bayes aplicado al latente.

EM alterna paso E (responsabilidades) y paso M (μ_k, Σ_k, π_k); hay máximos locales.

De la identidad al modelo

- $P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1})$ es exacta: es la regla del producto, no todavía un modelo
- El modelo aparece cuando se decide qué se hace con cada factor
- Si el factor depende solo del anterior: cadena de Markov de primer orden, que es el bigrama del notebook de hoy
- Si depende de los M anteriores: cadena de orden M, con más memoria y más parámetros
- Si se intercala una cadena de latentes discretos: modelo de espacio de estados, es decir un HMM, con transiciones A, inicio π y emisiones $p(x_n \mid z_n)$
- Si el factor se aprende con una red que ve todo el prefijo: modelo autorregresivo, el camino del LLM

Tres modelos, una identidad y tres suposiciones

$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1})$$

exacta: es la regla del producto, no todavía un modelo

el modelo aparece cuando se decide qué se hace con cada factor

truncar a un solo
factor anterior

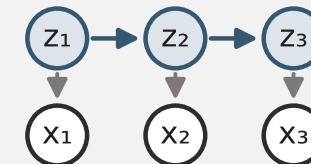
bigrama



cada factor mira
solo el anterior

intercalar una cadena
de latentes discretos

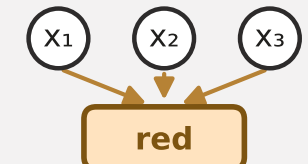
HMM



el estado latente resume
la historia

aprender el factor
con una red

LLM autorregresivo



el factor se aprende
sobre todo el prefijo

Los tres son la misma identidad con distintas suposiciones, no teorías rivales.

Bishop escribe esa factorización en 2006: no es una idea de los LLM. Lo nuevo es cómo se aprende el factor.

‘Un HMM solo recuerda el paso anterior.’ ¿Verdadero o falso?

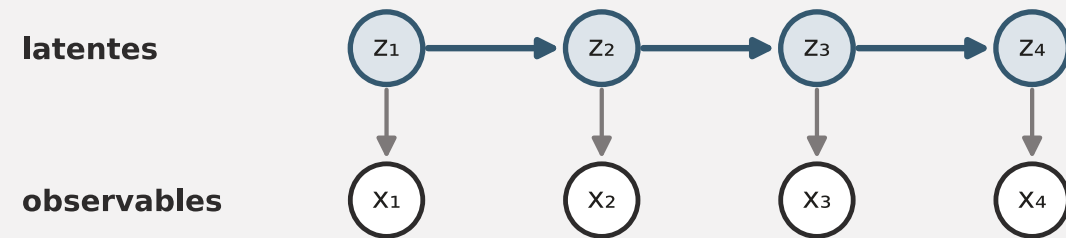
A. Verdadero: la cadena es de primer orden, así que la memoria es de un paso

B. Falso: $p(x_{n+1} \mid x_1, \dots, x_n)$ depende de todas las observaciones anteriores

C. Falso, pero solo cuando la matriz de transición no es dispersa

El grafo del HMM: latentes arriba, observables abajo

$z_1 \dots z_4$ son los estados latentes; $x_1 \dots x_4$, lo que se observa. Cada observación cuelga de su latente y los latentes se encadenan.



sobre este grafo: ¿qué camino une x_1 con x_4 , y qué tendría que pasar para que quedara bloqueado?



el estrechamiento

¿de qué depende?

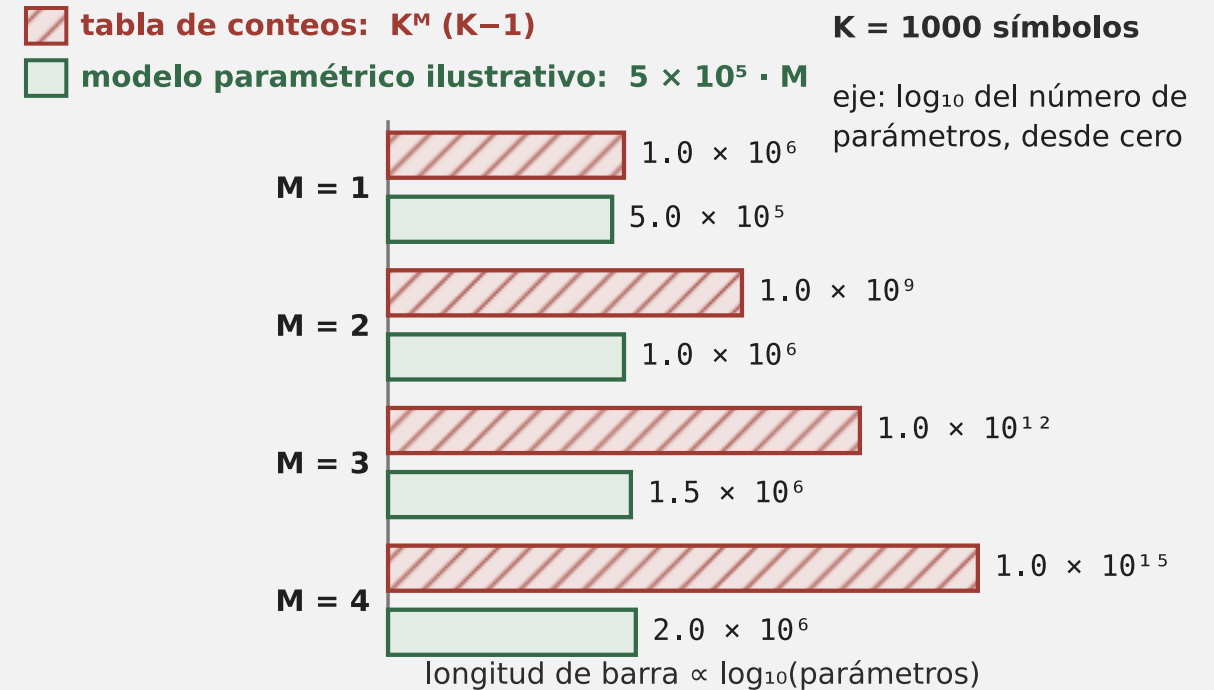
Las tres opciones se recogen antes de recorrer el grafo en voz alta.

La regla que decide si un camino queda bloqueado —la d-separación— está en Bishop (2006), §13.1, p. 610, y se lee después.

Contar no escala; compartir parámetros sí

- Una tabla de conteos de orden M sobre K símbolos necesita $K^M (K-1)$ parámetros
- Crece exponencialmente con M : subir el orden de la cadena es impracticable
- La alternativa está en el mismo libro, en 2006: usar un modelo paramétrico para $p(x_n \mid x_{n-M}, \dots, x_{n-1})$, ‘como una red neuronal’
- Con ese modelo el número de parámetros puede crecer linealmente con M , no exponencialmente
- Esa frase es el camino al LLM, y responde la pregunta 2 del notebook de hoy: la tabla no generaliza a pares no vistos

Contar no escala; compartir parámetros sí



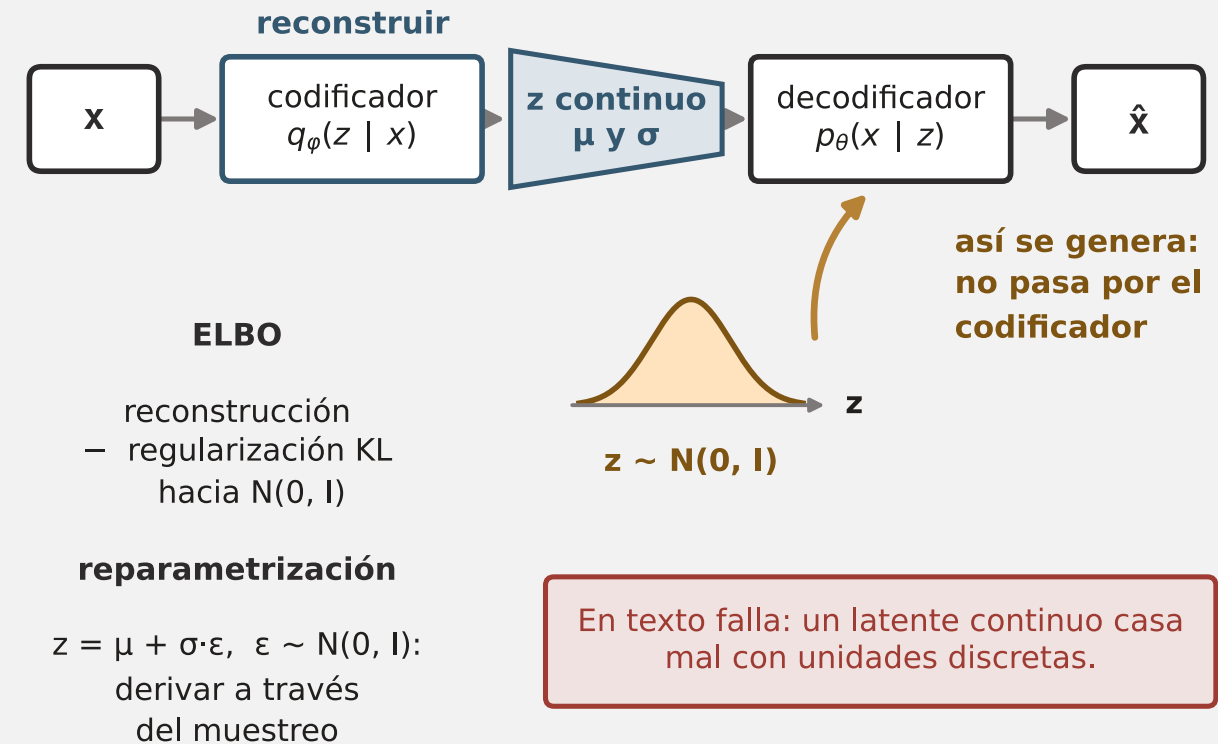
Subir el orden de la cadena multiplica los parámetros por K; el modelo paramétrico solo los suma.

La alternativa está en el mismo libro, en 2006: un modelo paramétrico para $p(x_n \mid x_{n-M}, \dots, x_{n-1})$, «como una red neuronal».

VAE, comprimir X en un latente continuo

- Codificador $q_\phi(z | x)$ entrega μ y σ^2 sobre un latente continuo; decodificador $p_\theta(x | z)$ reconstruye
- Objetivo (ELBO): reconstrucción menos una regularización KL que empuja el latente hacia $N(0, I)$
- Truco de reparametrización: $z = \mu + \sigma \cdot \varepsilon$ con $\varepsilon \sim N(0, I)$, para poder derivar a través del muestreo
- Genera muestreando $z \sim N(0, I)$ y decodificando: produce todo x de una vez, no pieza por pieza
- En texto falla: un latente continuo casa mal con unidades discretas y las muestras salen incoherentes
- Lo que deja: la idea de codificador-decodificador y la de espacio latente, primas del espacio de representaciones de la semana 2

Generar y reconstruir usan el mismo decodificador



Produce todo x de una vez, no pieza por pieza.

Lo que deja: codificador-decodificador y espacio latente, primas del espacio de representaciones de la semana 2.

Todo modelo generativo define $P(X)$

- Naive Bayes: $P(X, Y)$ factorizado · sin latente: Y se observa al entrenar y solo se muestrea al generar · limita la independencia condicional
- GMM: mezcla de gaussianas · latente z discreto · no capta estructura secuencial
- HMM: $P(x_1 \dots x_n)$ secuencial · latente z_n discreto · resume toda la historia en K estados
- VAE: $P(X)$ con codificador · latente z continuo · muestras incoherentes en texto discreto
- LLM: $P(x_t \mid x_{<t})$, sin latente explícita.
Autorregresivo: sus salidas previas son la entrada de la siguiente predicción
- Lo que cambia es cómo se factoriza $P(X)$ y a qué escala se aprende; el LLM lo paga en cómputo, que crece más rápido que la longitud del texto. La cuenta exacta, mañana

Los cinco responden la misma pregunta			
lo que cambia es cómo se factoriza $P(X)$ y a qué escala se aprende			
modelo	qué define	variable latente	limitación
Naive Bayes	$P(X, Y)$ factorizado	ninguna: Y se observa	la independencia condicional
GMM	mezcla de gaussianas	z discreta	no capta la estructura secuencial
HMM	$P(x_1 \dots x_n)$ secuencial	z_n discreta	resume toda la historia en K estados
VAE	$P(X)$ con codificador	z continua	muestras incoherentes en texto discreto
LLM	$P(x_t \mid x_{<t})$	ninguna explícita	el cómputo crece más rápido que la longitud

Autorregresivo: sus salidas previas son la entrada de la predicción siguiente.

Todo modelo generativo define $P(X)$: se distinguen por la factorización, no por el objetivo.

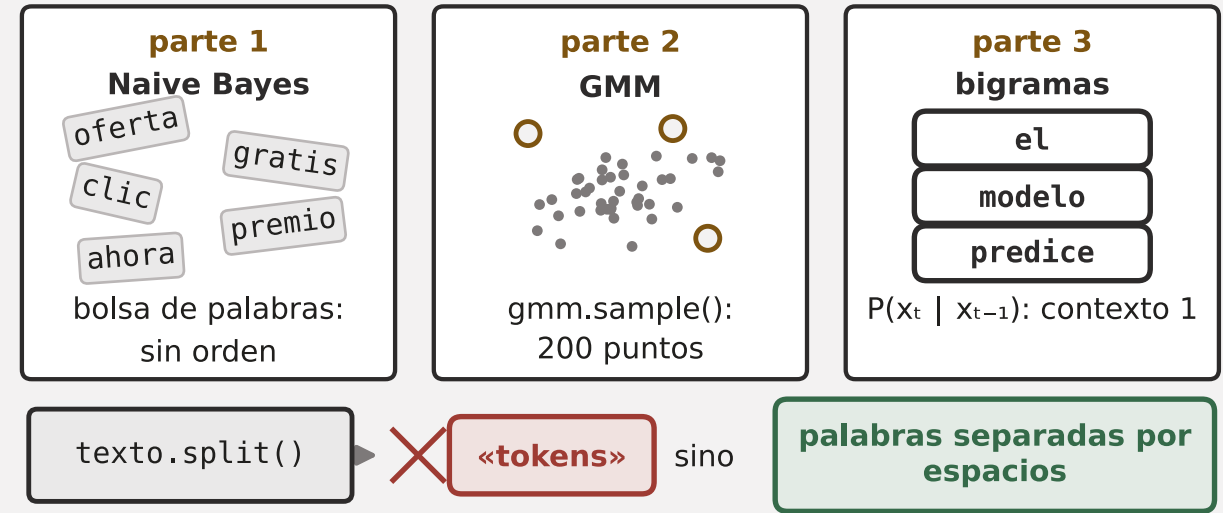
Entrenar un LLM maximiza $P(x_1, \dots, x_n)$ sobre mucho texto.

Fuente: fuentes/notas-teoricas/s1-transformers-y-foundation-models.md, §2.7 (esa tabla da «Clase Y» como variable latente de Naive Bayes; aquí se corrige, porque Y se observa al entrenar y §2.4 de la misma nota define latente como no observable); Raschka (2024), cap. 1, pp. 12–13

La actividad de hoy, y una advertencia de vocabulario

- Parte 1: entrenas un Naive Bayes sobre correos de juguete y lo usas al revés, muestreando de $P(\text{palabra} \mid \text{clase})$
- Parte 2: ajustas un GMM y generas 200 puntos nuevos con `gmm.sample()`, y los comparas con los reales
- Parte 3: cuentas bigramas y muestreas 20 pasos de $P(x_t \mid x_{t-1})$: un GPT de contexto 1
- Advertencia: el notebook llama 'tokens' al resultado de `texto.split()`. Partir por espacios fue durante décadas la tokenización más común, y es la de la bolsa de palabras con la que empezó esta historia
- Pero un token es una unidad del vocabulario del modelo, no una palabra: en chino no hay espacios que partir, y el costo de una API se factura en tokens. La tokenización real, mañana
- Corre local con numpy, scikit-learn y matplotlib. No necesita ninguna clave de API

Tres modelos generativos y una advertencia de vocabulario



Es una simplificación deliberada de la actividad, no de la definición: un token es una unidad del vocabulario del modelo, no una palabra.

La tokenización real se ve mañana. El costo de las APIs se calcula en tokens: estimarlos contando palabras da un costo equivocado.

Corre local con numpy, scikit-learn y matplotlib. No necesita ninguna clave de API.

El enunciado del notebook de esta noche

- 1.1 — Muestra `n_palabras` del vocabulario usando $P(\text{palabra} \mid \text{spam})$ como pesos, y genera un correo. Pista: `rng.choice` acepta `p=`
- 2.1 — Genera 200 puntos nuevos con `gmm.sample()` y gráfalos junto a los reales, en dos colores. ¿Se parecen las nubes?
- 3.1 — Construye la tabla de bigramas: `conteos[w1][w2]` es cuántas veces `w2` sigue a `w1`. Pista: `defaultdict(Counter)` sobre `zip(tokens, tokens[1:])`
- 3.2 — Genera texto: empieza en una palabra y muestra 20 pasos de $P(x_t \mid x_{t-1})$
- Cierre — ¿Qué le falta al bigrama para ser un GPT? Al menos tres cosas, en la celda del final. Se recoge mañana
- 45 minutos. Corre local con numpy, scikit-learn y matplotlib; no necesita ninguna clave de API

Cuatro entregables y una pregunta abierta

1.1 un correo muestreado de $P(\text{palabra} \mid \text{spam})$

2.1 200 puntos nuevos de `gmm.sample()`, sobre los reales

3.1 la tabla de conteos de bigramas

3.2 una cadena de 20 palabras muestreada paso a paso

¿Qué le falta al bigrama para ser un GPT? Al menos tres cosas, en la celda del final.

La única de la tarde que no se resuelve hoy: se recoge mañana.

45 minutos · corre local con numpy, scikit-learn y matplotlib · ninguna clave de API